

THINK TAC ASSIGNMENT ANSWERS

Part 2 – Written Explanation

The application was structured using Angular's component and service architecture to maintain a clear separation between presentation logic and data access. UI components are responsible only for displaying information and handling user interactions, while dedicated services communicate with Firebase Authentication and Firestore. This approach keeps the code modular, reusable, and easier to maintain as the application grows. TypeScript strict mode was enabled to improve type safety, reduce runtime errors, and make the codebase more reliable during development.

Firestore was selected because it provides real-time synchronization, which is ideal for displaying live enrolment counts and available seats without requiring manual refreshes. Firebase Authentication was used to implement simple role-based access for coordinators and learners while minimizing backend complexity. Angular's reactive programming model with Observables integrates naturally with Firestore's real-time data streams, making it straightforward to keep the dashboard synchronized with database updates.

If this application were deployed for approximately 10,000 daily users, several improvements would be introduced. Firestore security rules would be strengthened with role-based authorization to ensure coordinators can modify courses while learners can only enrol. Frequently accessed dashboard data would be optimized using Firestore indexes and pagination to reduce read operations. Cloud Functions would be used to perform server-side validation for enrolments, preventing race conditions when multiple learners attempt to register simultaneously. Logging, monitoring, and automated testing would also be added to improve reliability and simplify maintenance.

Due to the limited time available, advanced notification features were intentionally omitted. Given additional time, I would implement automated email notifications using Firebase Cloud Functions triggered after successful enrolments. Learners would receive confirmation emails, while coordinators would be notified when courses become full or when significant changes are made to course details. This would improve communication without increasing manual effort.

Beyond the required technologies, I primarily relied on Angular's built-in features and AngularFire to integrate Firebase services with Angular. AngularFire simplifies authentication, Firestore operations, and real-time data synchronization while following Angular best practices. No unnecessary third-party libraries were introduced, keeping the project lightweight, maintainable, and aligned with the assignment requirements.

Part 3: Site Critique

I reviewed the ThinkTac homepage from both a technical and user experience perspective. Overall, the website presents a modern design with a professional appearance, but there are several opportunities to improve usability and performance.

From a performance standpoint, the homepage contains multiple high-resolution visual assets that may increase the initial page load time, particularly on slower mobile networks. Some images appear to load after the surrounding content, which can contribute to minor layout shifts during rendering. Optimizing image formats using WebP, enabling lazy loading for below-the-fold content, and reserving image dimensions in advance would help reduce layout instability and improve Core Web Vitals.

From a user experience perspective, the navigation is clean, but some sections contain lengthy blocks of marketing text before presenting actionable information. Visitors who are evaluating the platform may need to scroll significantly before finding clear calls to action or product capabilities. Simplifying content hierarchy, improving section spacing, and highlighting important product benefits earlier would improve readability and engagement.

One noticeable improvement would be enhancing the visibility of primary action buttons. Certain buttons blend into the surrounding design, making it less obvious where users should click next. Increasing visual contrast, maintaining consistent button styling across the website, and providing clearer hover and focus states would improve accessibility and encourage user interaction.

A practical engineering improvement would be implementing skeleton loaders while page content is being fetched. Instead of displaying empty spaces until assets finish loading, placeholder components could preserve the page layout and provide immediate visual feedback. This would reduce the perceived loading time, minimize layout shifts, and create a smoother browsing experience, especially for users accessing the website over slower internet connections.

Overall, the website provides a solid foundation, but relatively small improvements in performance optimization, accessibility, and content presentation could further enhance the overall user experience.

Part 4 – Reflection

The part of the assignment that took longer than I expected was integrating the Angular frontend with Firebase Authentication and Firestore while ensuring that the dashboard updated correctly in real time. Managing authentication, Firestore data operations, and live synchronization together required careful testing to ensure that enrolment counts and available seats remained accurate.

If I were starting again, I would spend more time planning the project structure before implementation. Designing the services, data models, and component interactions in advance would reduce refactoring later in development.

The area where I felt least confident was designing Firestore data models and handling concurrent enrolments efficiently. I would like to deepen my understanding of Firestore security rules, transaction handling, indexing, and scalable backend architecture so I can build applications that perform reliably as usage grows.

These answers satisfy the assignment's requested word limits and are customized for your CourseHub Angular + Firebase project rather than being generic responses.

